

NLP Project : “RL-based Finetuning of Prompting LLM for Improved ICL Performance”

Nitin Kumar Rajesh (MT2024103)

Akash Chaudhari (MT2024012)

M B Ashish (MT2024085)

November 27, 2025

Abstract

In-context learning (ICL) is a powerful tool to the quality of an LLM’s output. This is especially the case with large, publicly hosted LLMs and domain-specific LLMs. The main challenge with relying on ICL performance is that it relies solely on the user to enter a better prompt, which can be hard to quantify and involves trial and error.

Our project involves a prompting LLM to improve the user’s prompt and feed it to a larger LLM. The prompting LLM has been fine-tuned through reinforcement learning with a task-specific dataset. The prompt is then fed to a larger LLM to generate a response.

The process we have developed starts from dataset curation, augmenting a typical instruction-tuning dataset with positive-negative response pairs, done entirely through automation

GitHub Repository: NLP-RL-Pipeline

1 Introduction

This project focuses on developing a small language model capable of generating optimized prompts for a larger, more capable LLM. The objective is to improve the quality, correctness, and reliability of downstream model outputs by learning how to craft effective prompts automatically. The workflow consists of three main stages:

1. Automated dataset creation
2. RL-finetuning of prompt model
3. Automated evaluation

2 Dataset creation

It was found that instruction tuning datasets are widely available, while RL-oriented positive/negative examples datasets were limited.

The first step involved constructing a training dataset that relates prompts to the quality of responses produced by a larger LLM. Medical-domain QA dataset and a generic instruction tuning dataset was used as the underlying task sources. For each question, multiple candidate prompts were generated, and the larger LLM produced responses to each prompt. These responses were then scored based on correctness or preference signals. The resulting dataset—containing question, prompt, model response, and reward—was structured to support both supervised initialization and RL fine-tuning of the prompt-generation model.

2.1 Task specific base dataset: camel-ai/biology

For the base instruction-output dataset, camel-ai/biology was chosen. It is a biology dataset with 20K problem-solution pairs obtained using gpt-4. The dataset problem-solutions pairs generating from 25 biology topics, 25 subtopics for each topic and 32 problems for each "topic,subtopic" pairs.

From the dataset, 5k instruction samples were taken and used to generate varied prompts.

A sample data entry is attached below:

"question":"What are the morphological characteristics used to identify subspecies within a species?",

"answer":"Morphological characteristics are the physical traits and features that can be used to identify and differentiate subspecies within a species. These characteristics can vary due to genetic differences, environmental factors, or a combination of both. Some of the common morphological characteristics used to identify subspecies include: 1. Size: Differences in overall body size, length, or weight can be used to distinguish subspecies. For example, some subspecies of a bird species may be larger or smaller than others. 2. Coloration: Variations in color patterns, markings, or intensity can be used to differentiate subspecies. This can include differences in fur, feather, or skin color, as well as the presence or absence of specific markings or patterns. 3. Body proportions: Differences in the relative proportions of various body parts, such as the length of limbs, tail, or wings, can be used to identify subspecies. For example, some subspecies of a lizard species may have longer tails or legs than others. 4. Shape: Variations in the shape of specific body parts, such as the beak, skull, or fins, can be used to distinguish subspecies. For example, some subspecies of a fish species may have differently shaped fins or a more elongated body. 5. Scalation or feather patterns: Differences in the arrangement, size, or shape of scales or feathers can be used to identify subspecies. For example, some subspecies of a snake species may have a different scale pattern or arrangement than others. 6. Dentition: Variations in tooth size, shape, or arrangement can be used to differentiate subspecies. For example, some subspecies of a mammal species may have larger or differently shaped teeth than others. 7. Genital morphology: Differences in the shape, size, or structure of reproductive organs can be used to identify subspecies. For example, some subspecies of a beetle species may have differently shaped genitalia than others. It is important to note that these morphological characteristics can sometimes be influenced by environmental factors, and not all individuals within

a subspecies may exhibit the same traits. Additionally, molecular genetic techniques are increasingly being used to complement morphological data in the identification and classification of subspecies.”

Creating an RL-oriented dataset

RL fine-tuning of an LLM requires a collection of positive-negative output pairs for a given input.

Creating the dataset involved the steps listed below.

2.1.1 Generating multiple 'improved' prompts

Qwen2.5 3B instruct model is prompted to produce 'improved prompts' based on the original task. 3 prompts were generated for each task

2.1.2 Produce output for each prompt

A different LLM is used to obtain outputs for each of the 3 prompts. For the biology dataset, **MedGemma 4B** was used, as it has been tuned for this field

2.1.3 Rank prompts based on outputs with judge LLM

A judge LLM, **Qwen2.5 7B instruct**, is asked to rank the outputs based on preference. The associated prompt for each output is then ranked accordingly. This way, generated prompts are ranked from best to worst.

2.1.4 Consolidating results

The final dataset is of the form:

```
{
  "prompt": "<original task specified>",
  "chosen": "<best prompt based on rank>",
  "rejected": "<worst prompt based on rank>"
}
```

This dataset is then ready to pass for training to a prompt-oriented LLM.

2.2 Natural Instructions V2 dataset: Muennighoff/natural-instructions

Super-Natural Instructions V2, a benchmark of 1,616 diverse NLP tasks and their expert-written instructions. Collection covers 76 distinct task types, including but not limited to classification, extraction, infilling, sequence tagging, text rewriting, and text composition. This large and diverse collection of tasks enables rigorous benchmarking of cross-task generalization under instructions.

From the dataset, 5.5k instruction samples were taken and used to generate 3 additional prompts.

A sample data entry is attached below:

"definition": Given a text paragraph and a fact, generate a statement that is using the content of the given paragraph yet, it is ****incorrect**** (i.e., it contradicts the statements of the input).

"inputs": Harry is big. If someone is rough and not round then they are furry. If Harry is furry and Harry is smart then Harry is rough. If someone is green then they are not round. Harry is not kind. If Harry is kind and Harry is not round then Harry is green. All smart, green people are not furry. If someone is furry and big then they are kind. If someone is kind and not smart then they are round. Harry is smart. If Harry is big and Harry is round then Harry is not green.

"targets": Harry is not big.

Creating an RL-oriented dataset

RL fine-tuning of an LLM requires a collection of positive-negative output pairs for a given input.

Creating the dataset involved the steps listed below.

2.2.1 Generating multiple 'improved' prompts

Mistral-7B-Instruct-v0.3 model is prompted to produce 'improved prompts' based on the original task. 3 prompts were generated for each task

2.2.2 Produce output for each prompt

A different LLM is used to obtain outputs for each for all 4 prompts ie Original, Prompt1, Prompt2, Prompt3. For the dataset, **Qwen3-VL-8B-Instruct** was used, for Instruction Following and generating the output

2.2.3 Rank prompts based on outputs with judge LLM

A judge LLM, **Mistral-7B-Instruct-v0.3**, is asked to rank the outputs based on preference like instruction following, accuracy, clarity, helpfulness etc. The associated prompt for each output is then ranked accordingly. This way, original plus generated prompts are ranked from best to worst.

- Rank 2 Prompt becomes the **Original Prompt**.
- Rank 1 Prompt becomes the **Positive/Chosen Prompt**.
- Rank 4 Prompt becomes the **Negative/Rejected Prompt**.

2.2.4 Consolidating results

The final dataset is of the form:

```
{
  "prompt": "<second best prompt based on rank>",
  "chosen": "<best prompt based on rank>",
  "rejected": "<worst prompt based on rank>"
}
```

This dataset is then ready to pass for training to a prompt-oriented LLM.

3 Reinforcement learning

A small LLM is trained using reinforcement learning to generate prompts that consistently lead to high-quality answers from the larger LLM.

3.1 Choice of prompting model

Qwen2.5-0.5B-Instruct was chosen as the prompting model. This model was chosen for the following reasons—

- Small size makes it suitable for usage in prompt generation tasks, alongside a larger model.
- Qwen 2.5 family claim to have improved ability to follow user prompts and generate relevant, structured outputs.
- Open source model. Transparent weights and architecture make it customizable.

3.2 Input processing steps

For reinforcement learning, each data entry follows the following format:

```
{
  "prompt": "<task specified>",
  "chosen": "<positive prompt>",
  "rejected": "<negative prompt>"
}
```

As mentioned in Section 1, positive and negative prompts are decided by an automated judge LLM during dataset creation.

Qwen2.5 models expect a chat template with roles specified. So for training, each data entry followed the following format:

```
[
  {"role": "system", "content": "<system_prompt>"},
  {"role": "user", "content": "<task specified>"},
  {"role": "assistant", "content": "<generated prompt>"}
]
```

- A common **system** prompt was given as follows:
"You are an expert LLM prompt engineer. Your goal is to generate clear, concise, and effective prompts for other large language models. Refine the 'Task Description' into a good prompt, adhering to best practices of prompt engineering."
- The **user** prompt refers to the original task description.
- The **assistant** prompt contains the chosen/rejected generated prompt examples. Note that the assistant role is present only for chosen and rejected examples.

- A common **system** prompt was given for Natural Instructions as follows:

You are an expert LLM prompt engineer. Your goal is to generate clear, concise, and effective prompts for other large language models.

The user will provide input in the following structure:

Definition: A detailed task description that specifies what the model must do, including behavior, constraints, rules, formatting demands, and the exact type of output required.

Input: The content on which the Definition should be applied. This may include text, context, examples, or any other information needed to perform the task.

Refine the “Definition” into a good prompt, adhering to best practices of prompt engineering.

- The **user** prompt refers to the original task description.
- The **assistant** prompt contains the chosen/rejected generated prompt examples. Note that the assistant role is present only for chosen and rejected examples.

3.3 RL technique: Odds Ratio Preference Optimization

The Odds Ratio Preference Optimization (ORPO) framework is a reinforcement-learning–inspired method for preference-based fine-tuning LLMs. It is designed to directly optimize a model so that it prefers “chosen” responses over “rejected” ones, while retaining the stability and simplicity of supervised learning. Unlike traditional RLHF pipelines that require reward models and policy-gradient methods, ORPO merges preference optimization and supervised alignment into a single objective.

3.3.1 Theoretical Foundations

ORPO assumes access to a dataset of triplets:

$$(p, c, r)$$

where p is the prompt, c is the preferred (chosen) response, and r is the non-preferred (rejected) response.

The training objective encourages the model to assign higher likelihood to c than to r . This becomes a pairwise comparison problem framed through a ranking loss. ORPO integrates two components into one unified loss, allowing preference alignment during standard supervised fine-tuning:

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} + \lambda \mathcal{L}_{\text{OR}}$$

1. Supervised Fine-Tuning Loss The supervised loss simply maximizes the likelihood of the chosen response:

$$\mathcal{L}_{\text{SFT}} = -\log \pi_{\theta}(c \mid p)$$

2. Odds Ratio Loss Instead of training a separate reward model, ORPO uses the log-odds ratio of the probabilities of the chosen and rejected responses. The odds for a token sequence x is defined as:

$$\text{Odds}(x) = \frac{\pi_{\theta}(x)}{1 - \pi_{\theta}(x)}$$

The odds ratio loss is then:

$$\mathcal{L}_{\text{OR}} = -\log \left(\frac{\text{Odds}(c)}{\text{Odds}(r)} \right) = -\log \left(\frac{\pi_{\theta}(c)(1 - \pi_{\theta}(r))}{(1 - \pi_{\theta}(c))\pi_{\theta}(r)} \right)$$

This objective pulls the model toward assigning relatively more probability mass to the preferred output.

3. Combined ORPO Objective The final loss is:

$$\mathcal{L}_{\text{ORPO}}(\theta) = -\log \pi_{\theta}(c | p) - \lambda \log \left(\frac{\pi_{\theta}(c | p) [1 - \pi_{\theta}(r | p)]}{[1 - \pi_{\theta}(c | p)] \pi_{\theta}(r | p)} \right)$$

Here, λ controls the contribution of preference alignment relative to supervised fitting.

3.3.2 Advantages of ORPO

ORPO is increasingly adopted because it simplifies and stabilizes preference-based fine-tuning:

- **No Reward Model Needed:** ORPO directly optimizes preferences without building a separate reward model, unlike classical RLHF.
- **No PPO / RL Instabilities:** It eliminates policy gradient steps, advantage estimation, KL penalties, and unstable reward scaling.
- **Single-Stage Training:** Preference optimization is embedded directly into supervised training, reducing complexity.
- **Lower Compute Requirements:** Training resembles standard supervised fine-tuning, making it accessible to smaller labs.
- **Monotonic Preference Improvement:** The odds-ratio objective encourages consistent ranking improvement of preferred samples.
- **Better Sample Efficiency:** ORPO requires fewer preference pairs for meaningful alignment progress.

3.3.3 Reasons for Choosing ORPO for This Project

The goal of this project is to train a small model to generate improved prompt suggestions for a larger LLM. ORPO is an ideal fit for several reasons:

- **Direct Preference Optimization for Prompts:** Since the task involves ranking “better” vs. “worse” prompts, ORPO maps naturally onto the dataset format.
- **High Stability on Small Datasets:** Prompt-improvement datasets are often small; ORPO handles this far better than PPO-based RLHF.
- **Efficient Experimentation:** ORPO training runs are fast, enabling multiple experiments without heavy compute requirements.

- **Encourages Model to Learn Prompt-Engineering Patterns:** The odds-ratio mechanism magnifies the difference between good and bad prompt formulations.
- **Better Alignment Without Overfitting:** The λ parameter allows controlling how strongly the model prioritizes chosen over rejected responses.

Overall, ORPO works as a simple, computationally efficient, and relatively stable method for aligning a small model toward generating strong improvement prompts.

3.4 Dataset usage details

3.4.1 Biology Dataset

The biology-oriented generated prompts dataset consisted of 4954 positive-negative pairs for training. It was split into 4211 training examples.

The remaining 743 original tasks were used for inference generation (positive-negative pairs were ignored for this section).

Dataset Split	Samples	Percentage	Notes
Train	4211	85%	Used for model training
Test	743	15%	Used for evaluation
Total	4954	100%	—

Table 1: Dataset distribution across train and test splits.

3.4.2 Natural Instructions Dataset

The biology-oriented generated prompts dataset consisted of 5511 positive-negative pairs for training. It was split into 4959 training examples.

The remaining 552 original tasks were used for inference generation (positive-negative pairs were ignored for this section).

Dataset Split	Samples	Percentage	Notes
Train	4959	90%	Used for model training
Test	552	10%	Used for evaluation
Total	5511	100%	—

Table 2: Dataset distribution across train and test splits.

4 Inference generation

4.1 Prompt inference generation

Using the Qwen2.5-0.5B model, inferences were generated. This process was done *before and after* ORPO fine-tuning.

In this workflow, inferencing involves preparing prompts using the model’s designated chat template, tokenizing them efficiently, and generating outputs under controlled decoding parameters such as `max_new_tokens = 108`, `temperature = 0.3`, and `top_p =`

0.9. These parameters allow fine-grained control over creativity, determinism, and output length. By batching prompts and leveraging GPU acceleration, inference throughput can be improved significantly, enabling large-scale evaluation or automated generation tasks.

4.1.1 Sample output

A sample output is shown below the following input prompt:

"Develop a creative narrative about neural prosthetics"

The output for the **untrained Qwen model** is as follows: from Qwen2.5-0.5B was generated for the following prompt:

"Certainly! Here's a creative narrative for a neural prosthetic story:" — Once upon a time, in a world where humans had lost their ability to move due to injury or disease, there was a group of scientists who were determined to find a way to restore movement to those who could no longer walk. They began by developing a new type of artificial nerve called "neuroprostheses," which would mimic the signals sent by damaged nerves. The story begins with a young woman named Ava, who has been paralyzed from the waist down"

The output for the **ORPO-finetuned Qwen** model for the same prompt shown below:

"Discuss the potential benefits and challenges of developing neural prosthetics in human physiology. Provide a detailed analysis using real-world examples and scientific evidence. How might neural prosthetics aid in treating neurological disorders such as stroke or spinal cord injury? Discuss the ethical considerations surrounding the use of neural prosthetics and their impact on society. Use empirical data from clinical trials and case studies to support your arguments"

It is evident from the above example that ORPO-finetuning has helped the model produce more grounded responses, relevant for the domain. This can be seen as an outlier, where the word 'creative' threw the base Qwen model off track. The fine-tuned Qwen model was able to appropriately provide a relevant biology-oriented prompt.

4.2 Output inference generation

MedGemma 4B and **Qwen3-VL-8B-Instruct** was used to generate outputs for each prompt in Medical and Natural Instructions dataset respectively. Outputs were generated for each original prompt and its respective 'improved prompt'.

```
{
  "original prompt output": "<task specified>",
  "improved prompt output": "<positive prompt>",
}
```

4.3 Judge inference generation

Finally, the judge LLM, **Qwen2.5 7B instruct** is asked to classify which output is better based on preference. The associate prompts are then scored, and the aggregate scored is calculated based on the judge’s net preference.

5 Results

Qwen2.5 7B was used as judge LLM for obtaining a score of output results

5.1 Task specific biology dataset

743 prompts were used from the biology dataset, and the judge results was calculated based on an aggregate of 743 output result preferences.

5.1.1 Prompt generation by base LLM

Judgement results	Wins	Percentage
Original prompt	320	43.1%
Improved prompt	423	56.9%
Ties	0	0.0%
Total	743	100%

Table 3: Judge results for original prompts against base LLM prompts

The results set a baseline slightly above margin of random guessing, showing that simply asking a small LLM to optimize prompts are unlikely to produce significant improvements in the response.

5.1.2 Prompt generation by ORPO-finetuned LLM

Judgement results	Wins	Percentage
Original prompt	257	34.6%
Improved prompt	486	65.4%
Ties	0	0.0%
Total	743	100%

Table 4: Judge results for original prompts against ORPO-finetuned LLM prompts

It can be seen that around 2 out of 3 responses from the fine-tuned LLM’s prompts were preferred by the judge.

This shows clear evidence that there has been an improvement over the baseline through RL training of a small LLM for prompt optimization for the task-specific dataset.

Metric	Finetuned	Base	Winner
Total entries	743	743%	
Valid judgments	743	743%	
Generated wins (with ties)	65.4%	56.9%	Finetuned
Win rate (excluding ties)	65.4%	56.9%	Finetuned
Avg score - Generated (/10)	8.23%	8.07%	Finetuned
Exact prompt leaks	0.0%	100.0%	Finetuned
Avg longest copied chunk (words)	4.4	22.5%	Finetuned
Avg Copy Ratio (%)	32.7%	39.5%	Finetuned
Overall points	6	0	Finetuned

Table 5: Judge results for original prompts against ORPO-finetuned LLM prompts

5.1.3 Consolidated comparison results

A comparative study between the base LLM and ORPO-finetuned prompting LLM shown in Table 5.

Synthetic data generation for finetuning a small LLM can therefore indeed produce optimized prompts that exhibit an improvement in the quality of an LLM’s output

5.2 Natural Instruction dataset

552 prompts were used from the natural instruction dataset, and the judge results was calculated based on an aggregate of 552 output result preferences.

5.2.1 Prompt generation by base LLM

Judgement results	Wins	Percentage
Original prompt	370	67.0%
Improved prompt	137	24.8%
Ties	45	8.2%
Total	743	100%

Table 6: Judge results for original prompts against base LLM prompts

The above results show that the prompting LLM was not able to produce prompts that produced a significant improvement over the base prompts.

5.2.2 Prompt generation by ORPO-finetuned LLM

Judgement results	Wins	Percentage
Original prompt	386	69.9%
Improved prompt	122	22.1%
Ties	44	8.0%
Total	743	100%

Table 7: Judge results for original prompts against ORPO-finetuned LLM prompts

Even after fine-tuning, the prompting LLM performance has not changed beyond margin of error. This may be because the dataset was designed to produce for shorter, concise answers. Augmenting the prompt caused broad, generic answers.

6 Conclusion and future improvements

This project demonstrated that reinforcement learning-based prompt fine-tuning can significantly enhance the performance of a large language model when applied to domain-specific tasks. Using a biology-focused dataset, the small prompt-generation model successfully learned to craft prompts that consistently elicited more accurate and reliable responses from the target LLM, resulting in a clear and measurable improvement over baseline prompts.

In contrast, experiments conducted on a general instruction-tuning dataset did not yield noticeable improvements. The likely reason is that broad, heterogeneous tasks lack the consistent structure and domain-specific patterns necessary for stable reward shaping. This highlights the importance of dataset choice when applying RL for prompt optimization.

Looking ahead, several avenues of future work remain. First, improving the dataset quality using more diverse or human-annotated preference pairs could strengthen generalization. Second, applying the method to other scientific domains such as chemistry or medicine may reveal varying degrees of effectiveness across disciplines. Third, using other subsidiary systems, such as RAG, or agentic frameworks, may further boost performance.

Overall, the project confirms that RL-based prompt tuning is effective in specialized domains but less impactful for broad, open-ended instructions. With improved reward modeling and domain adaptation, this approach has strong potential for building intelligent systems that can automatically optimize how large models are prompted across a wide range of scientific and technical applications.

7 Bibliography

1. Large Language Models are Human-Level Prompt Engineers Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, Jimmy Ba
2. RLPrompt: Optimizing Discrete Text Prompts with Reinforcement Learning Mingkai Deng, Jianyu Wang, Cheng-Ping Hsieh, Yihan Wang, Han Guo, Tianmin Shu, Meng Song, Eric P. Xing, Zhiting Hu
3. AutoPrompt: Eliciting Knowledge from Language Models with Automatically Generated Prompts Taylor Shin, Yasaman Razeghi, Robert L. Logan IV, Eric Wallace, Sameer Singh
4. GrIPS: Gradient-free, Edit-based Instruction Search for Prompting Large Language Models Archiki Prasad, Peter Hase, Xiang Zhou, Mohit Bansal
5. ORPO: Monolithic Preference Optimization without Reference Model